INTERVIEW PREPARATION SERIES

# Weighted Graphs, MST, Prim's & Kruskal's

LeetCode Problems & Solutions — 10 Problems

This practice set covers **10 curated LeetCode problems** on Weighted Graphs, MST, Prim's & Kruskal's: Min Cost Connect Points, Network Delay Time, Cheapest Flights K Stops, City Threshold Distance, Max Probability Path, Redundant Connection, Connect Cities Min Cost, Swim in Rising Water, Connected Components, and Graph Valid Tree. Each problem shows the companies that frequently ask it, complete solutions in Python, Java, and C++, and a structured 4-tier breakdown. Solve in any language — the concepts are universal.

PROBLEM 01

## Min Cost to Connect All Points Medium

▶ **Asked at:** **Amazon** (2024) • **Google** (2024) • **Microsoft** (2023)

▶ **Frequency:** High — 600+ interview occurrences, the canonical MST problem on LeetCode

### TIER 1 — UNDERSTAND

Given n points on a 2D plane, return the minimum cost to connect all points where cost = Manhattan distance. This is MST on a complete graph.

### TIER 2 — APPROACH

**Prim's $O(V^2)$:** dense complete graph ($n^2$ edges), so Prim's without a heap is optimal. Track minimum distance from each unvisited node to the MST.

### TIER 3 — ALGORITHM SKETCH

```
# Python (Prim's O(V^2))
def minCostConnectPoints(points):
    n = len(points)
```

```
    visited = [False]*n; min_d = [float('inf')]*n; min_d[0]=0; total=0
    for _ in range(n):
        u = min((i for i in range(n) if not visited[i]), key=lambda i: min_d[i])
        visited[u] = True; total += min_d[u]
        for v in range(n):
            if not visited[v]:
                d = abs(points[u][0]-points[v][0])+abs(points[u][1]-points[v][1])
                min_d[v] = min(min_d[v], d)
    return total
```

```
// Java
public int minCostConnectPoints(int[][] p) {
    int n=p.length, total=0; boolean[] vis=new boolean[n];
    int[] dist=new int[n]; Arrays.fill(dist, Integer.MAX_VALUE); dist[0]=0;
    for (int i=0;i<n;i++) {
        int u=-1;
        for (int j=0;j<n;j++) if (!vis[j]&&(u==-1||dist[j]<dist[u])) u=j;
        vis[u]=true; total+=dist[u];
        for (int v=0;v<n;v++) if (!vis[v]) {
            int d=Math.abs(p[u][0]-p[v][0])+Math.abs(p[u][1]-p[v][1]);
            dist[v]=Math.min(dist[v],d);
        }
    } return total;
}
```

### TIER 4 — COMPLEXITY

**Time:** $O(n^2)$ | **Space:** O(n)

**Interview tip:** this is a **dense** graph ($n^2$ edges). Prim's $O(V^2)$ beats Kruskal's $O(E \log E) = O(n^2 \log n)$ here.

---

### PROBLEM 02

## Network Delay Time Medium

▶ **Asked at:** **Google** (2024) • **Amazon** (2024) • **Meta** (2023)

▶ **Frequency:** High — 500+ interview occurrences, the canonical Dijkstra problem

### TIER 1 — UNDERSTAND

Given a directed weighted graph of n nodes, send a signal from node k. Return the time for all nodes to receive it (max of shortest paths), or -1 if not all reachable.

### TIER 2 — APPROACH

Dijkstra's from source k. Return max distance if all nodes reachable, else -1.

### TIER 3 — ALGORITHM SKETCH

```
# Python
import heapq
def networkDelayTime(times, n, k):
    graph = [[] for _ in range(n+1)]
    for u,v,w in times: graph[u].append((v,w))
    dist = [float('inf')]*(n+1); dist[k]=0
    heap = [(0, k)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]: continue
        for v, w in graph[u]:
            if dist[u]+w < dist[v]:
                dist[v] = dist[u]+w
                heapq.heappush(heap, (dist[v], v))
    mx = max(dist[1:])
    return mx if mx < float('inf') else -1
```

### TIER 4 — COMPLEXITY

**Time:** O(E log V) | **Space:** O(V + E)

**Interview tip:** Dijkstra's cannot handle negative weights. If asked about negatives, use Bellman-Ford.

---

### PROBLEM 03

## Cheapest Flights Within K Stops Medium

▶ **Asked at:** **Amazon** (2024) • **Google** (2024) • **Bloomberg** (2024)

▶ **Frequency:** High — 500+ interview occurrences, tests BFS/Bellman-Ford with constraints

### TIER 1 — UNDERSTAND

Find cheapest flight from src to dst with at most k stops. Standard Dijkstra doesn't work because it doesn't track stops. Tests modified BFS or Bellman-Ford.

### TIER 2 — APPROACH

**Bellman-Ford with k+1 iterations:** relax all edges k+1 times (k stops = k+1 edges max). Each iteration extends paths by one edge.

### TIER 3 — ALGORITHM SKETCH

```
# Python (Bellman-Ford variant)
```

```
def findCheapestPrice(n, flights, src, dst, k):
    dist = [float('inf')] * n; dist[src] = 0
    for _ in range(k + 1):
        tmp = dist[:]
        for u, v, w in flights:
            if dist[u] + w < tmp[v]:
                tmp[v] = dist[u] + w
        dist = tmp
    return dist[dst] if dist[dst] < float('inf') else -1
```

TIER 4 — COMPLEXITY

**Time:** O(k × E) | **Space:** O(V)

**Interview tip:** must copy dist array each iteration (`tmp = dist[:]`) to prevent using updates from the same round.

PROBLEM 04

## Find the City With the Smallest Number of Neighbors at a Threshold Distance Medium

▶ **Asked at:** **Google** (2023) • **Amazon** (2024)

▶ **Frequency:** Medium — 300+ interview occurrences, tests Floyd-Warshall or repeated Dijkstra

TIER 1 — UNDERSTAND

Find the city with the fewest reachable cities within a distance threshold. Ties broken by largest city number. Requires all-pairs shortest paths.

TIER 2 — APPROACH

**Floyd-Warshall O($V^3$):** compute all-pairs shortest paths, then count reachable cities per node.

TIER 3 — ALGORITHM SKETCH

```
# Python
def findTheCity(n, edges, threshold):
    dist = [[float('inf')]*n for _ in range(n)]
    for i in range(n): dist[i][i] = 0
    for u,v,w in edges: dist[u][v]=dist[v][u]=w
    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(dist[i][j], dist[i][k]+dist[k][j])
```

```
    best, min_count = -1, n+1
    for i in range(n):
        count = sum(1 for j in range(n) if i!=j and dist[i][j]<=threshold)
        if count <= min_count: min_count=count; best=i
    return best
```

### TIER 4 — COMPLEXITY

**Time:** $O(V^3)$ | **Space:** $O(V^2)$

**Interview tip:** Floyd-Warshall is simple but $O(V^3)$. For large V, run Dijkstra from each node: O(V × E log V).

---

PROBLEM 05

## Path With Maximum Probability Medium

▶ **Asked at:** **Google** (2024) • **Amazon** (2023)

▶ **Frequency:** Medium — 300+ interview occurrences, tests Dijkstra variant with max instead of min

### TIER 1 — UNDERSTAND

Find the path with maximum success probability from start to end. Edge weights are probabilities (0 to 1). Multiply probabilities along the path. This is Dijkstra inverted: maximize instead of minimize.

### TIER 2 — APPROACH

**Max-heap Dijkstra:** use a max-heap (negate probabilities for Python's min-heap). Relax edges by checking if `prob[u] * w > prob[v]`.

### TIER 3 — ALGORITHM SKETCH

```
# Python
def maxProbability(n, edges, succProb, start, end):
    graph = [[] for _ in range(n)]
    for (u,v), p in zip(edges, succProb):
        graph[u].append((v,p)); graph[v].append((u,p))
    prob = [0.0]*n; prob[start]=1.0
    heap = [(-1.0, start)]  # max-heap via negation
    while heap:
        p, u = heapq.heappop(heap); p = -p
        if u == end: return p
        if p < prob[u]: continue
        for v, w in graph[u]:
```

```
            if p*w > prob[v]:
                prob[v] = p*w
                heapq.heappush(heap, (-prob[v], v))
    return 0.0
```

### TIER 4 — COMPLEXITY

**Time:** O(E log V) | **Space:** O(V + E)

**Interview tip:** any "maximize product along path" can be converted to "minimize sum" by taking -log of weights. Then standard Dijkstra works.

---

PROBLEM 06

## Redundant Connection Medium

▶ **Asked at:** **Amazon** (2024) • **Google** (2024) • **Meta** (2023)

▶ **Frequency:** High — 500+ interview occurrences, the canonical Union-Find problem

### TIER 1 — UNDERSTAND

A tree of n nodes has one extra edge creating a cycle. Find and return the extra edge. This is: find the first edge that connects two already-connected nodes.

### TIER 2 — APPROACH

**Union-Find:** process edges one by one. The first edge where `find(u) == find(v)` (both already in same component) is the redundant edge.

### TIER 3 — ALGORITHM SKETCH

```
# Python
def findRedundantConnection(edges):
    uf = UnionFind(len(edges) + 1)
    for u, v in edges:
        if not uf.union(u, v):
            return [u, v]  # this edge creates a cycle
```

```
// Java
public int[] findRedundantConnection(int[][] edges) {
    int n = edges.length;
    int[] parent = new int[n+1], rank = new int[n+1];
    for (int i=0;i<=n;i++) parent[i]=i;
    for (int[] e : edges)
        if (!union(parent,rank,e[0],e[1])) return e;
    return new int[]{};
```

```
}
```

TIER 4 — COMPLEXITY

**Time:** O(n × α(n)) ≈ O(n) | **Space:** O(n)

**Interview tip:** Union-Find with path compression + rank is near O(1) per operation. This pattern appears in MST, connected components, and graph validation.

PROBLEM 07

## Connecting Cities With Minimum Cost Medium

▶ **Asked at:** **Amazon** (2024) • **Microsoft** (2023)

▶ **Frequency:** Medium — 400+ interview occurrences, direct MST application

TIER 1 — UNDERSTAND

Given n cities and weighted connections, find minimum cost to connect all cities. Return -1 if impossible. This is MST — use Kruskal's since we have an edge list.

TIER 2 — APPROACH

**Kruskal's:** sort edges by cost, Union-Find to add edges without cycles, stop when V−1 edges added.

TIER 3 — ALGORITHM SKETCH

```
# Python
def minimumCost(n, connections):
    connections.sort(key=lambda x: x[2])
    uf = UnionFind(n + 1)
    total, edges_added = 0, 0
    for u, v, w in connections:
        if uf.union(u, v):
            total += w; edges_added += 1
            if edges_added == n - 1: return total
    return -1  # not all cities connected
```

TIER 4 — COMPLEXITY

**Time:** O(E log E) | **Space:** O(V)

**Interview tip:** Kruskal's is preferred when edges are given as a list (not adjacency). Early termination at V−1 edges saves work.

PROBLEM 08

## Swim in Rising Water Hard

▶ **Asked at:** **Google** (2024) • **Amazon** (2024)

▶ **Frequency:** Medium — 300+ interview occurrences, tests minimax path thinking

### TIER 1 — UNDERSTAND

In an n×n grid of elevations, find the minimum time T such that you can swim from (0,0) to (n-1,n-1) where at time T all cells with elevation ≤ T are underwater. This is a **minimax path** problem: minimize the maximum elevation on the path.

### TIER 2 — APPROACH

**Modified Dijkstra:** instead of summing weights, track the maximum elevation on the path. Use a min-heap keyed by max elevation so far.

### TIER 3 — ALGORITHM SKETCH

```
# Python
def swimInWater(grid):
    n = len(grid)
    heap = [(grid[0][0], 0, 0)]
    visited = {(0,0)}
    while heap:
        t, r, c = heapq.heappop(heap)
        if r==n-1 and c==n-1: return t
        for dr,dc in [(0,1),(0,-1),(1,0),(-1,0)]:
            nr, nc = r+dr, c+dc
            if 0<=nr<n and 0<=nc<n and (nr,nc) not in visited:
                visited.add((nr,nc))
                heapq.heappush(heap, (max(t, grid[nr][nc]), nr, nc))
```

### TIER 4 — COMPLEXITY

**Time:** $O(n^2 \log n)$ | **Space:** $O(n^2)$

**Interview tip:** this is the **minimax path** pattern. Instead of summing costs, take the max. Related to minimum bottleneck spanning tree.

PROBLEM 09

## Number of Connected Components in Undirected Graph Medium

▶ **Asked at:** **Amazon** (2024) • **Meta** (2024) • **LinkedIn** (2023)

▶ **Frequency:** High — 600+ interview occurrences, fundamental Union-Find application

### TIER 1 — UNDERSTAND

Given n nodes and an edge list, count connected components. Direct application of Union-Find.

### TIER 2 — APPROACH

**Union-Find:** start with n components. Each successful union reduces count by 1. Final count = number of components.

### TIER 3 — ALGORITHM SKETCH

```python
# Python
def countComponents(n, edges):
    uf = UnionFind(n)
    for u, v in edges: uf.union(u, v)
    return uf.components  # or: len(set(uf.find(i) for i in range(n)))
```

```java
// Java
public int countComponents(int n, int[][] edges) {
    int[] parent = new int[n]; for (int i=0;i<n;i++) parent[i]=i;
    int count = n;
    for (int[] e : edges) {
        int pu=find(parent,e[0]), pv=find(parent,e[1]);
        if (pu!=pv) { parent[pu]=pv; count--; }
    }
    return count;
}
```

### TIER 4 — COMPLEXITY

**Time:** O(E × α(V)) ≈ O(E) | **Space:** O(V)

**Interview tip:** Union-Find is better than DFS when edges arrive as a stream (online). DFS needs the full graph upfront.

---

PROBLEM 10

## Graph Valid Tree Medium

▶ **Asked at:** **Amazon** (2024) • **Google** (2023) • **Meta** (2024)

▶ **Frequency:** High — 500+ interview occurrences, combines Union-Find with tree properties

### TIER 1 — UNDERSTAND

Given n nodes and edges, determine if they form a valid tree. A tree is: connected, acyclic, and has exactly n−1 edges.

### TIER 2 — APPROACH

Check: (1) exactly n−1 edges, (2) no cycle (Union-Find: every union must succeed). If both pass, it's a valid tree.

### TIER 3 — ALGORITHM SKETCH

```python
# Python
def validTree(n, edges):
    if len(edges) != n - 1: return False
    uf = UnionFind(n)
    for u, v in edges:
        if not uf.union(u, v): return False  # cycle
    return True
```

```cpp
// C++
bool validTree(int n, vector<vector<int>>& edges) {
    if (edges.size() != n-1) return false;
    vector<int> parent(n);
    iota(parent.begin(), parent.end(), 0);
    for (auto& e : edges) {
        int pu=find(parent,e[0]), pv=find(parent,e[1]);
        if (pu==pv) return false;
        parent[pu]=pv;
    }
    return true;
}
```

### TIER 4 — COMPLEXITY

**Time:** O(E × α(V)) ≈ O(E) | **Space:** O(V)

**Interview tip:** the edge count check (n−1) is a quick reject. Combined with cycle detection, it fully validates a tree.

## SUMMARY TABLE

| # | Problem | Diff | Companies | Key Pattern | Topic |
|---|---|---|---|---|---|
| 1 | Min Cost Connect Points | Medium | Amazon, Google, Microsoft | Prim's $O(V^2)$ | MST |
| 2 | Network Delay Time | Medium | Google, Amazon, Meta | Dijkstra | Shortest Path |
| 3 | Cheapest Flights K Stops | Medium | Amazon, Google, Bloomberg | Bellman-Ford | Shortest Path |
| 4 | City Threshold Dist | Medium | Google, Amazon | Floyd-Warshall | All-Pairs |
| 5 | Max Probability Path | Medium | Google, Amazon | Max Dijkstra | Shortest Path |
| 6 | Redundant Connection | Medium | Amazon, Google, Meta | Union-Find | Cycle Detection |
| 7 | Connect Cities Min Cost | Medium | Amazon, Microsoft | Kruskal's | MST |
| 8 | Swim in Rising Water | Hard | Google, Amazon | Minimax Dijkstra | MST Variant |
| 9 | Connected Components | Medium | Amazon, Meta, LinkedIn | Union-Find | Components |
| 10 | Graph Valid Tree | Medium | Amazon, Google, Meta | UF + Edge Count | Tree Validation |